

Kierunek Informatyka

Systemy wbudowane
Rok akademicki 2025/2026, semestr letni

Sprawozdanie z projektu semestralnego

Nr projektu: **XX**

Tytuł projektu:
Projekt i implementacja elektrycznego pieca sklepowego w oparciu o
emulator PICSimLab

Wykonawca:
Atlas

Data złożenia: **11/06/2026**

Spis treści

1. Specyfikacja wymagań projektowych
2. Konfiguracja sprzętowa
3. Diagram przedstawiający algorytm działania systemu
4. Szczegółowy opis implementacji

1. Specyfikacja wymagań projektowych

1.1. Problematyka zagadnienia

Celem projektu jest implementacja uproszczonego sterownika elektrycznego pieca sklepowego z wykorzystaniem mikrokontrolera **PIC16F628A** oraz środowiska symulacyjnego **PICSimLab**. System pozwoli na wybór programu pieczenia, kontrolowanie temperatury i czasu pieczenia, obsługę sytuacji awaryjnych, obsługę debouncingu oraz automatyczne mycie pieca.

1.2. Koncepcja rozwiązania

W celu stworzenia sterownika obsługującego elektryczny piec sklepowy - założono wykorzystanie języka **C**, środowiska **MPLAB X IDE**, kompilatora **Microchip XC8** i emulatora **PICSimLab**. Ze względu to iż mikrokontroler, na którym ma być uruchamiany sterownik ma swoje ograniczenia sprzętowe (ograniczona pamięć i stos) to wykorzystano dyrektywę **#define** czyli makro oraz słowo kluczowe **inline**. Podanie słowa kluczowego **inline** powoduje, że wstawienie kodu funkcji w miejscu wywołania jest preferowane nad typowym mechanizmem wywoływania funkcji.

Ze względu na znajomość ograniczeń programowania strukturalnego, które mogą powodować problemy z dalszą rozbudową programu – wykorzystano model maszyny stanowej, który znacznie uprości proces tworzenia oprogramowania.

Projektowany system wbudowany będzie działać w jedenastu stanach:

- STATE_POWERUP,
- STATE_DISABLED,
- STATE_READY,
- STATE_HEATING,
- STATE_INSERT_CARTRIDGE,
- STATE_INSERT_PRODUCTS,
- STATE_BAKING,
- STATE_GET_PRODUCTS,
- STATE_PREPARE_TO_CLEANING,
- STATE_CLEANING,
- STATE_ERROR,

System umożliwi użytkownikowi sterowanie za pomocą przycisków podłączonych do portu **A** mikrokontrolera, natomiast sygnalizacja stanu maszyny odbywać się będzie przy użyciu lampy (**RA0**), diod LED (**RB0-RB7**) podłączonych do portu **B** oraz wyświetlacza siedmiosegmentowego. Całość kodu będzie obsługiwać także przerwania względem elementów **TMRO** i **TMR1**, które odpowiadają za odmierzenie czasu i wyświetlanie wartości na wyświetlaczu siedmiosegmentowym.

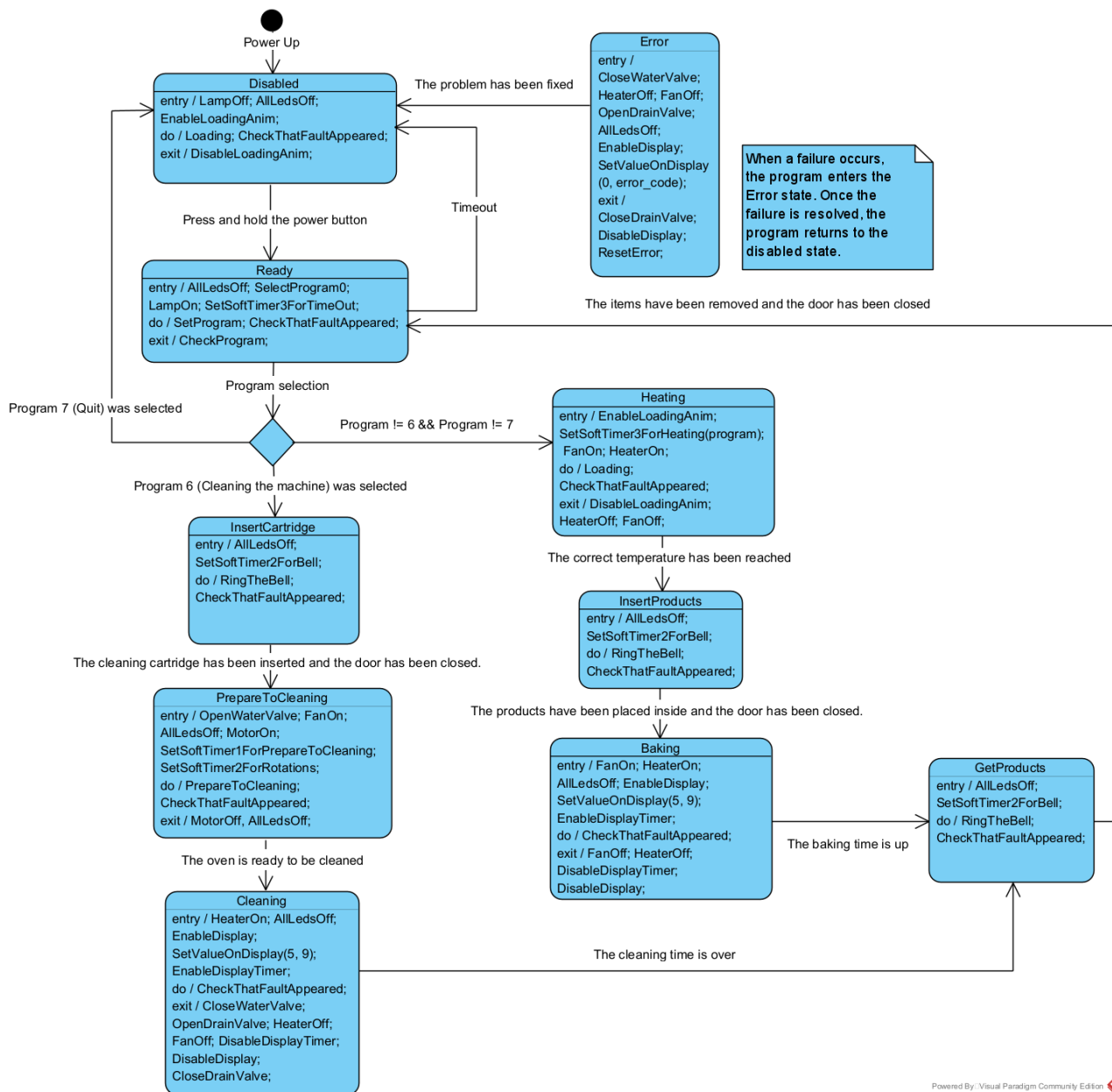
2. Konfiguracja sprzętowa

Nazwa	Rodzaj	Opis
Port A		
RA0	wyjście cyfrowe	Włączenie/wyłączenie lampy sygnalizującej gotowość pieca.
RA1	wejście cyfrowe	Przycisk włączający piec (PowerButton)/przycisk przełączający na wyższy program (UpperButton).
RA2	wejście cyfrowe	Przycisk przełączający na niższy program (LowerButton).
RA3	wejście cyfrowe	Przycisk zatwierdzający (OkButton).
RA4	wejście cyfrowe	Przycisk symulujący wystąpienie awarii (FaultSimulationButton).
RA5	-	nieużywany
RA6	-	nieużywany
RA7	-	nieużywany
Port B		
Cały port B jest przeznaczony również do sygnalizowania: - aktualnego stanu animacji odblokowania pieca (diody od RB7 do RB0 zapalają się po kolei), - poziomu nagrzania pieca (diody od RB7 do RB0 zapalają się po kolei), - stanu przygotowania do czyszczenia pieca (pomiędzy diodami RB0 do RB7 krąży sygnał). - dzwonka, który oznacza pewne zdarzenia (diody od RB0 do RB7 zapalają się i gasną naprzemiennie): - gotowość pieca na włożenie produktów do odpieku, - gotowość pieca na wyjęcie gotowych produktów z pieca, - gotowość pieca na włożenie wkładu do czyszczenia pieca, - gotowość pieca na wyjęcie wykorzystanego wkładu do czyszczenia pieca, - port B jest też wykorzystywany do wyświetlenia pozostałego czasu pracy lub kodu błędu na wyświetlaczu siedmiosegmentowym,		
RB0	wyjście cyfrowe	- W stanie gotowości sygnalizuje wybrany program 0.
RB1	wyjście cyfrowe	- Dioda sygnalizująca włączenie/wyłączenie silnika, który obraca ramieniem natryskowym; - W stanie gotowości sygnalizuje wybrany program 1.
RB2	wyjście cyfrowe	- W stanie gotowości sygnalizuje wybrany program 2.
RB3	wyjście cyfrowe	- W stanie gotowości sygnalizuje wybrany program 3.
RB4	wyjście cyfrowe	- Dioda sygnalizująca otwarcie/zamknięcie zaworu wody (otwiera/zamyka elektrozawór, wpuszczając wodę do komory). - W stanie gotowości sygnalizuje wybrany program 4.
RB5	wyjście cyfrowe	- Dioda sygnalizująca otwarcie/zamknięcie zaworu odpływowego wody (steruje odpływem wody i chemii do kanalizacji). - W stanie gotowości sygnalizuje wybrany program 5.
RB6	wyjście cyfrowe	- Dioda sygnalizująca włączenie/wyłączenie wentylatora (włącza/wyłącza obieg powietrza). - W stanie gotowości sygnalizuje wybrany program 6.
RB7	wyjście cyfrowe	- Dioda sygnalizująca włączenie/wyłączenie grzałki (włącza/wyłącza przełącznik podającym prąd na elektryczne elementy grzejne). - W stanie gotowości sygnalizuje wybrany program 7.

Port B będzie wykorzystywany do sygnalizowania pewnych zdarzeń za pomocą diod LED oraz wyświetlania informacji na wyświetlaczu siedmiosegmentowym. Wyświetlacz siedmiosegmentowy w czasie odpieku lub czyszczenia sygnalizuje pozostały czas pracy. Wyświetla on również kod błędu w przypadku awarii.

3. Diagram maszyny stanowej

Diagram maszyny stanowej przedstawia stany systemu, jego zmiany i zachowanie. Na rysunku 1 przedstawiono diagram maszyny stanowej wykonany w programie Visual Paradigm Community Edition.



Rysunek 1. Diagram maszyny stanowej elektrycznego pieca sklepowego.

W przedstawionej maszynie stanowej wyróżniamy następujące stany z odpowiednimi opisami:

- STATE_POWERUP – Tymczasowy stan startowy, który od razu przechodzi w stan zablokowany.
- STATE_DISABLED – Stan, w którym piec jest zablokowany. W celu przejścia w stan gotowości, należy przytrzymać przycisk PowerButton.
- STATE_READY – W tym stanie można wybrać odpowiedni program pracy pieca.
- STATE_HEATING – Stan nagrzewania pieca.

- STATE_INSERT_CARTRIDGE – W tym stanie umieszcza się specjalny wkład czyszczący. Po włożeniu wkładu zatwierdzamy jego włożenie i maszyna przechodzi w stan przygotowania do czyszczenia.
- STATE_INSERT_PRODUCTS – W tym stanie wkłada się do pieca produkty do odpieku.
- STATE_BAKING – Stan pieczenia produktów.
- STATE_GET_PRODUCTS – Stan odbioru gotowych produktów lub pustego wkładu czyszczącego.
- STATE_PREPARE_TO_CLEANING – Przygotowanie do czyszczenia pieca.
- STATE_CLEANING – Stan czyszczenia pieca.
- STATE_ERROR – W przypadku wystąpienia błędu - program przechodzi w stan awarii. Po usunięciu usterki, program powraca do stanu zablokowanego (STATE_DISABLED).

Oprócz powyższych stanów wyróżniamy również stany pośrednie/akcje:

- ACTION_ENTRY – Akcja wykonywana jeden raz podczas wejścia do stanu.
- ACTION_DO – Akcja wykonywana wiele razy w trakcie przebywania w danym stanie, cyklicznie powtarzana do momentu zajścia warunku wyjścia.
- ACTION_EXIT – Akcja wykonywana jeden raz podczas wyjścia ze stanu.

4. Szczegółowy opis implementacji

W tym punkcie przedstawiono i opisano szczegółowe znaczenie poszczególnych fragmentów kodu źródłowego aplikacji:

a) newmain.c

```
1. // Config:
2. // -----
3. #pragma config FOSC    = EXTRCCLK // Oscillator Selection bits (RC oscillator:
   CLKOUT function on RA6/OSC2/CLKOUT pin, Resistor and Capacitor on RA7/OSC1/CLKIN)
4. #pragma config WDTE    = OFF // Watchdog Timer Enable bit (WDT disabled)
5. #pragma config PWRTE   = OFF // Power-up Timer Enable bit (PWRT disabled)
6. #pragma config MCLRE   = ON // RA5/MCLR/VPP Pin Function Select bit (RA5/MCLR/VPP
   pin function is MCLR)
7. #pragma config BOREN   = ON // Brown-out Detect Enable bit (BOD enabled)
8. #pragma config LVP     = ON // Low-Voltage Programming Enable bit (RB4/PGM pin has PGM
   function, low-voltage programming enabled)
9. #pragma config CPD     = OFF // Data EE Memory Code Protection bit (Data memory code
   protection off)
10. #pragma config CP      = OFF // Flash Program Memory Code Protection bit (Code
   protection off)
11.
12. // Includes:
13. // -----
14. #include <xc.h>
15. #include <stdbool.h>
16. #include "Leds.h"
17. #include "StatesAndActions.h"
18. #include "Microcontroller.h"
19. #include "Programs.h"
20. #include "Errors.h"
21.
22. // Consts and macros:
23. // -----
24.
25. #define _XTAL_FREQ 1000000UL
26.
27.
28. // State machine - state functions def:
29. // -----
30.
31. void StatePowerUpFn(void)
32. {
33.     ChangeMachineState(STATE_DISABLED, StateDisabledFn); // Przejscie do stanu
   zablokowania
34.
35.     action = ACTION_ENTRY;
36. }
37.
38. void StateDisabledFn(void)
39. {
40.     switch(action)
41.     {
42.         case ACTION_ENTRY:
43.             LampOff();
44.             AllLedsOff();
45.             EnableLoadingAnim();
46.             action = ACTION_DO;
47.             break;
48.
49.         case ACTION_DO:
50.             //
51.             // PowerButton to RA1
52.             // W celu wlaczenia pieca nalezy przytrzymac RA1
53.             //
54.             if (IsPowerButtonPressed())
55.             {
```

```

56.             Loading();
57.             if(soft_timer_3 == 0)
58.             {
59.                 action = ACTION_EXIT;
60.             }
61.         }
62.     else
63.     {
64.         AllLedsOff();
65.         SetSoftTimer3ForLoading();
66.     }
67.
68.     CheckThatFaultAppeared();
69.
70.     break;
71.
72.     case ACTION_EXIT:
73.         DisableLoadingAnim();
74.
75.         ChangeMachineState(STATE_READY, StateReadyFn);
76.
77.         action = ACTION_ENTRY;
78.     break;
79. }
80. }
81.
82. void StateReadyFn(void)
83. {
84.     switch(action)
85.     {
86.         case ACTION_ENTRY:
87.             AllLedsOff();
88.             SelectProgram0();
89.             LampOn();
90.             action = ACTION_DO;
91.
92.             SetSoftTimer3ForTimeOut();
93.         break;
94.
95.         case ACTION_DO:
96.             //
97.             // RA1 przelacza na program wyzej
98.             // natomiast RA2 przelacza na program nizej
99.             // RA3 zatwierdza program
100.            //
101.            //
102.            if(SetProgram() == PROGRAM_SETTED)
103.            {
104.                ResetAllButtons();
105.                SetSoftTimer3ForTimeOut();
106.            }
107.
108.            if(soft_timer_3 == 0)
109.            {
110.                ChangeMachineState(STATE_DISABLED,
StateDisabledFn);
111.
112.                action = ACTION_ENTRY;
113.            }
114.
115.            CheckThatFaultAppeared();
116.
117.        break;
118.
119.        case ACTION_EXIT:
120.            //
121.            // Program 7 czyli led RB7 to jest wyjście
122.            // czyli powrót do stanu disabled
123.            //
124.            if(program == PROGRAM_7_QUIT)
125.            {

```



```

126.                                     ChangeMachineState (STATE_DISABLED,
StateDisabledFn);
127.
128.                                     action = ACTION_ENTRY;
129.                                     }
130.                                     //
131.                                     // Program 6 czyli led RB6 to jest mycie pieca
132.                                     //
133.                                     else if(program == PROGRAM_6_CLEANING_THE_MACHINE)
134.                                     {
135.                                     ChangeMachineState (STATE_INSERT_CARTRIDGE,
StateInsertCartridgeFn);
136.
137.                                     action = ACTION_ENTRY;
138.                                     }
139.                                     else
140.                                     {
141.                                     ChangeMachineState (STATE_HEATING,
StateHeatingFn);
142.
143.                                     action = ACTION_ENTRY;
144.                                     }
145.                                     break;
146.                                     }
147.                                     }
148.
149. void StateHeatingFn(void)
150. {
151.     switch(action)
152.     {
153.         case ACTION_ENTRY:
154.             action = ACTION_DO;
155.             EnableLoadingAnim();
156.             SetSoftTimer3ForHeating(program);
157.             FanOn();
158.             HeaterOn();
159.             break;
160.
161.         case ACTION_DO:
162.             //
163.             // Jak sie zaladuja lampki na full
164.             // czyli piec osiagnie odpowiednia temperature
165.             // to wtedy przechodzi do stanu wkladania
166.             // produktow do wypiekania
167.             //
168.             Loading();
169.             if(soft_timer_3 == 0)
170.             {
171.                 action = ACTION_EXIT;
172.             }
173.
174.             CheckThatFaultAppeared();
175.
176.             break;
177.
178.         case ACTION_EXIT:
179.             DisableLoadingAnim();
180.             HeaterOff();
181.             FanOff();
182.
183.             ChangeMachineState (STATE_INSERT_PRODUCTS,
StateInsertProductsFn);
184.
185.             action = ACTION_ENTRY;
186.             break;
187.         }
188.     }
189.
190. void StateInsertProductsFn(void)
191. {
192.     switch(action)

```

```

193.         {
194.             case ACTION_ENTRY:
195.                 AllLedsOff();
196.                 SetSoftTimer2ForBell();
197.                 action = ACTION_DO;
198.             break;
199.
200.             case ACTION_DO:
201.                 //
202.                 // W tym stanie lampki migaja w celu
203.                 // zasymulowania dzwonka.
204.                 //
205.                 // Wcisniecie przycisku RB3 czyli OK
206.                 // zatwierdza ze produkty zostaly wlozone
207.                 // i drzwiczki zostaly zamkniete
208.                 //
209.                 if(IsOkButtonPressed())
210.                 {
211.                     ResetAllButtons();
212.                     action = ACTION_EXIT;
213.                 }
214.                 else if (soft_timer_2 == 0)
215.                 {
216.                     RingTheBell();
217.                     SetSoftTimer2ForBell();
218.                 }
219.
220.                 CheckThatFaultAppeared();
221.             break;
222.
223.             case ACTION_EXIT:
224.
225.                 ChangeMachineState(STATE_BAKING, StateBakingFn);
226.
227.                 action = ACTION_ENTRY;
228.             break;
229.         }
230.     }
231.
232. void StateBakingFn(void)
233. {
234.     switch(action)
235.     {
236.         case ACTION_ENTRY:
237.             FanOn();
238.             HeaterOn();
239.             AllLedsOff();
240.             EnableDisplay();
241.             SetValueOnDisplay(5, 9);
242.             EnableDisplayTimer();
243.             action = ACTION_DO;
244.         break;
245.
246.         case ACTION_DO:
247.             //
248.             // Podczas pieczenia nalezy przelaczyc z ledow
249.             // na display
250.             //
251.             // Po zakonczeniu odliczania stan pieczenia przechodzi
252.             // w stan odbioru
253.             //
254.             if (IsDisplayTimerFinished())
255.             {
256.                 action = ACTION_EXIT;
257.             }
258.
259.             CheckThatFaultAppeared();
260.         break;
261.
262.         case ACTION_EXIT:
263.             FanOff();

```

```

264.             HeaterOff();
265.             DisableDisplayTimer();
266.             DisableDisplay();
267.
268.             ChangeMachineState(STATE_GET_PRODUCTS,
StateGetProductsFn);
269.
270.             action = ACTION_ENTRY;
271.             break;
272.         }
273.     }
274.
275. void StateGetProductsFn(void)
276. {
277.     switch(action)
278.     {
279.         case ACTION_ENTRY:
280.             AllLedsOff();
281.             SetSoftTimer2ForBell();
282.             action = ACTION_DO;
283.             break;
284.
285.         case ACTION_DO:
286.             //
287.             // W tym stanie lampki migaja w celu
288.             // zasymulowania dzwonka.
289.             //
290.             // Wcisniecie przycisku RB3 czyli OK
291.             // zatwierdza ze produkty zostaly wyjete
292.             // i drzwiczki zostaly zamkniete
293.             //
294.             if(IsOkButtonPressed())
295.             {
296.                 ResetAllButtons();
297.                 action = ACTION_EXIT;
298.             }
299.             else if (soft_timer_2 == 0)
300.             {
301.                 RingTheBell();
302.                 SetSoftTimer2ForBell();
303.             }
304.
305.             CheckThatFaultAppeared();
306.             break;
307.
308.         case ACTION_EXIT:
309.
310.             ChangeMachineState(STATE_READY, StateReadyFn);
311.
312.             action = ACTION_ENTRY;
313.             break;
314.         }
315.     }
316.
317. void StateInsertCartridgeFn(void)
318. {
319.     switch(action)
320.     {
321.         case ACTION_ENTRY:
322.             AllLedsOff();
323.             SetSoftTimer2ForBell();
324.             action = ACTION_DO;
325.             break;
326.
327.         case ACTION_DO:
328.             //
329.             // W tym stanie lampki migaja w celu
330.             // zasymulowania dzwonka.
331.             //
332.             // Wcisniecie przycisku RB3 czyli OK
333.             // zatwierdza ze wkład do czyszczenia zostal wlozony

```

```

334.                // i drzwiczki zostaly zamkniete
335.                //
336.                if(IsOkButtonPressed())
337.                {
338.                    ResetAllButtons();
339.                    action = ACTION_EXIT;
340.                }
341.                else if (soft_timer_2 == 0)
342.                {
343.                    RingTheBell();
344.                    SetSoftTimer2ForBell();
345.                }
346.
347.                CheckThatFaultAppeared();
348.            break;
349.
350.            case ACTION_EXIT:
351.
352.                ChangeMachineState(STATE_PREPARE_TO_CLEANING,
353.                StatePrepareToCleaningFn);
354.
355.                action = ACTION_ENTRY;
356.            break;
357.        }
358.
359.        void StatePrepareToCleaningFn(void)
360.        {
361.            switch(action)
362.            {
363.                case ACTION_ENTRY:
364.                    OpenWaterValve();
365.                    FanOn();
366.                    AllLedsOff();
367.                    MotorOn();
368.                    SetSoftTimer1ForPrepareToCleaning();
369.
370.                    SetSoftTimer2ForRotations();
371.
372.                    action = ACTION_DO;
373.                break;
374.
375.                case ACTION_DO:
376.                    //
377.                    // Czekamy az piec przygotowuje sie do mycia
378.                    // czyli jak led przestanie krazyc...
379.                    //
380.                    if (soft_timer_1 == 0) // soft timer 1 for preparing
381.                    cleaning
382.                    {
383.                        action = ACTION_EXIT;
384.                    }
385.                    if (soft_timer_2 == 0) // soft timer 2 for rotations
386.                    {
387.                        PrepareToCleaning();
388.
389.                        SetSoftTimer2ForRotations();
390.                    }
391.
392.                    CheckThatFaultAppeared();
393.                break;
394.
395.                case ACTION_EXIT:
396.                    MotorOff();
397.                    AllLedsOff();
398.
399.                    ChangeMachineState(STATE_CLEANING, StateCleaningFn); //
400.                    Przejscie do wlasciwej fazy mycia
401.
402.                    action = ACTION_ENTRY;
403.                break;

```

```

402.         }
403.     }
404.
405. void StateCleaningFn(void)
406. {
407.     switch(action)
408.     {
409.         case ACTION_ENTRY:
410.             HeaterOn();
411.             AllLedsOff();
412.             EnableDisplay();
413.             SetValueOnDisplay(5, 9);
414.             EnableDisplayTimer();
415.             action = ACTION_DO;
416.         break;
417.
418.         case ACTION_DO:
419.             //
420.             // Podczas mycia nalezy przelaczyc z ledow
421.             // na display
422.             //
423.             // Po zakonczeniu odliczania stan mycia przechodzi
424.             // w stan odbioru wkladu do mycia
425.             //
426.             if (IsDisplayTimerFinished())
427.             {
428.                 action = ACTION_EXIT;
429.             }
430.
431.             CheckThatFaultAppeared();
432.         break;
433.
434.         case ACTION_EXIT:
435.             CloseWaterValve();
436.             OpenDrainValve();
437.             HeaterOff();
438.             FanOff();
439.             DisableDisplayTimer();
440.             DisableDisplay();
441.             CloseDrainValve();
442.
443.             ChangeMachineState(STATE_GET_PRODUCTS,
StateGetProductsFn); // Zakladamy ze odbior wkladu i odbior gotowego produktu to
jedno i to samo
444.             // w celu zaoszczedzenia pamieci programu
445.             action = ACTION_ENTRY;
446.         break;
447.     }
448. }
449.
450. void StateErrorFn(void)
451. {
452.     switch(action)
453.     {
454.         case ACTION_ENTRY:
455.             DisableDisplayTimer();
456.             CloseWaterValve();
457.             HeaterOff();
458.             FanOff();
459.             OpenDrainValve();
460.             AllLedsOff();
461.             EnableDisplay();
462.             SetValueOnDisplay(0, error_code);
463.             action = ACTION_DO;
464.         break;
465.
466.         case ACTION_DO:
467.             if (IsOkButtonPressed())
468.             {
469.                 ResetAllButtons();
470.                 action = ACTION_EXIT;

```

```

471.         }
472.         break;
473.
474.         case ACTION_EXIT:
475.             CloseDrainValve();
476.             DisableDisplay();
477.             ResetError();
478.
479.             ChangeMachineState(STATE_DISABLED, StateDisabledFn); //
Powrot do stanu zablokowanego
480.
481.             action = ACTION_ENTRY;
482.             break;
483.         }
484.     }
485.
486.     // Entry:
487.     // -----
488.
489.     void main()
490.     {
491.         InitPeripherals(); // konfiguracja mikrokontrolera
492.
493.         while (1)
494.         {
495.             CurrentStateFunction();
496.         }
497.     }

```

Szczegółowy opis kodu z pliku **newmain.c**:

Linie 2-9:	Deklaracje konfigurujące tryb pracy mikrokontrolera – tzw. bity konfiguracyjne.
Linia 14:	Plik nagłówkowy kompilatora XC8.
Linie 15-20:	Plik nagłówkowy zawierający deklaracje standardowych funkcji bibliotecznych.
Linie 25:	Deklaracja częstotliwości taktowania mikrokontrolera (dla funkcji <code>__delay_ms()</code>).
Linie 31-36:	Definicja funkcji reprezentującej stan <code>STATE_POWERUP</code> .
Linia 33:	Przejście w stan zablokowania <code>STATE_DISABLED</code> .
Linie 38-80:	Definicja funkcji reprezentującej stan <code>STATE_DISABLED</code> .
Linie 49-70:	W celu włączenia pieca należy przytrzymać RA1. Jeśli wszystkie diody się zapalą to włączenie pieca sygnalizowane jest włączoną lampą RA0.
Linie 72-78:	Zmiana stanu ze stanu zablokowanego na stan gotowości <code>STATE_READY</code> .
Linie 82-147:	Definicja funkcji reprezentującej stan <code>STATE_READY</code> .
Linie 95-117:	W celu przełączania programów należy wciskać RA1 lub RA2. Zatwierdzenie programu odbywa się za pomocą przycisku RA3.
Linie 120-145:	Zmiana stanu w zależności od wybranego programu.
Linie 149-188:	Definicja funkcji reprezentującej stan <code>STATE_HEATING</code> .
Linie 161-187:	Gdy piec osiągnie odpowiednią temperaturę to dochodzi do zmiany stanu na stan wkładania produktów do pieca <code>STATE_INSERT_PRODUCTS</code> .
Linie 190-230:	Definicja funkcji reprezentującej stan <code>STATE_INSERT_PRODUCTS</code> .
Linie 200-221:	Jeśli włożono już produkty do pieca to wciśnięcie przycisku RB3 (OkButton) zatwierdza, że produkty zostały włożone i drzwiczki zostały zamknięte.
Linie 223-228:	Zmiana stanu na stan pieczenia <code>STATE_BAKING</code> .
Linie 232-273:	Definicja funkcji reprezentującej stan <code>STATE_BAKING</code> .
Linie 246-271:	Jeśli stoper wyświetlający na wyświetlaczu siedmiosegmentowym zakończy odliczanie to stan maszyny jest zmieniany na stan odebrania produktów <code>STATE_GET_PRODUCTS</code> .
Linie 275-315:	Definicja funkcji reprezentującej stan <code>STATE_GET_PRODUCTS</code> .
Linie 285-313:	W celu zasygnalizowania wyjęcia produktów z pieca i zamknięcia drzwiczek – wciskamy przycisk RA3 (OkButton). Po wciśnięciu przycisku, maszyna stanowa zmienia stan na stan gotowości <code>STATE_READY</code> .
Linie 317-357:	Definicja funkcji reprezentującej stan <code>STATE_INSERT_CARTRIDGE</code> .
Linie 327-348:	Jeśli włożono już wkład do czyszczenia to wciśnięcie przycisku RB3 (OkButton) zatwierdza, że wkład został włożony i drzwiczki zostały zamknięte.
Linie 359-403:	Definicja funkcji reprezentującej stan <code>STATE_PREPARE_TO_CLEANING</code> .
Linie 375-401:	Jeśli piec będzie gotowy do czyszczenia (LED przestanie krążyć i stoper zakończy odliczanie) to maszyna zmieni stan na stan mycia <code>STATE_CLEANING</code> .
Linie 405-448:	Definicja funkcji reprezentującej stan <code>STATE_CLEANING</code> .

- Linie 418-446: Jeśli piec zakończy mycie (stoper odliczający zakończy odliczanie) to maszyna zmienia stan na stan odbioru produktów `STATE_GET_PRODUCTS`.
- Linie 450-484: Definicja funkcji reprezentującej stan `STATE_ERROR`.
- Linie 452-482: Jeśli doszło do awarii pieca to na wyświetlaczu siedmiosegmentowym wyświetlany jest kod błędu. Po naprawie pieca należy wcisnąć RA3 (OkButton) w celu powrotu do stanu zablokowania `STATE_DISABLED`.
- Linie 489-497: Definicja funkcji głównej `main()`.
- Linia 491: Konfiguracja mikrokontrolera.
- Linia 495: Obsługa bieżącego stanu za pomocą wywołania funkcji spod wskaźnika.

b) Buttons.h

```
1. #ifndef BUTTONS_H
2. #define BUTTONS_H
3.
4. #include <xc.h>
5.
6. // Macros for buttons:
7. // -----
8.
9. #define BUTTON_ON          0    // Na płytce Board 1 przyciski pracuje w
    odwróconej logice
10. #define BUTTON_OFF        1    // czyli: 0 gdy przycisk jest wciśnięty, 1 gdy
    puszczony
11.
12. #define CHECK_MSEC         5    // Read hardware every 5 msec
13. #define PRESS_MSEC        20    // Stable time before registering pressed
14.
15. // Buttons:
16. // -----
17.
18. volatile static _Bool bt1_pressed = false;
19. volatile static _Bool bt2_pressed = false;
20. volatile static _Bool bt3_pressed = false;
21. volatile static _Bool bt4_pressed = false;
22.
23. // Check buttons without debouncing:
24. // -----
25.
26. // private:
27. // -----
28. inline int Private_IsButton1PressedWithoutDebouncing()
29. {
30.     return PORTAbits.RA1 == BUTTON_ON;
31. }
32.
33. inline int Private_IsButton2PressedWithoutDebouncing(void)
34. {
35.     return PORTAbits.RA2 == BUTTON_ON;
36. }
37.
38. inline int Private_IsButton3PressedWithoutDebouncing(void)
39. {
40.     return PORTAbits.RA3 == BUTTON_ON;
41. }
42.
43. inline int Private_IsButton4PressedWithoutDebouncing(void)
44. {
45.     return PORTAbits.RA4 == BUTTON_ON;
46. }
47. // -----
48.
49. // Debouncing for buttons:
50. // -----
51.
52. // private:
53. // -----
54. static void Private_DebounceButton1()
55. {
56.     static unsigned char counter = PRESS_MSEC/CHECK_MSEC;
57.
58.     if(Private_IsButton1PressedWithoutDebouncing())
59.     {
60.         if(--counter == 0)
61.             bt1_pressed = true;
62.     }
63.     else
64.     {
65.         counter = PRESS_MSEC/CHECK_MSEC;
```

```

66.         bt1_pressed = false;
67.     }
68. }
69.
70. static void Private_DebounceButton2()
71. {
72.     static unsigned char counter = PRESS_MSEC/CHECK_MSEC;
73.     if(Private_IsButton2PressedWithoutDebouncing())
74.     {
75.         if(--counter == 0)
76.             bt2_pressed = true;
77.     }
78.     else
79.     {
80.         counter = PRESS_MSEC/CHECK_MSEC;
81.         bt2_pressed = false;
82.     }
83. }
84.
85. static void Private_DebounceButton3()
86. {
87.     static unsigned char counter = PRESS_MSEC/CHECK_MSEC;
88.
89.     if(Private_IsButton3PressedWithoutDebouncing())
90.     {
91.         if(--counter == 0)
92.             bt3_pressed = true;
93.     }
94.     else
95.     {
96.         counter = PRESS_MSEC/CHECK_MSEC;
97.         bt3_pressed = false;
98.     }
99. }
100.
101. static void Private_DebounceButton4()
102. {
103.     static unsigned char counter = PRESS_MSEC/CHECK_MSEC;
104.
105.     if(Private_IsButton4PressedWithoutDebouncing())
106.     {
107.         if(--counter == 0)
108.             bt4_pressed = true;
109.     }
110.     else
111.     {
112.         counter = PRESS_MSEC/CHECK_MSEC;
113.         bt4_pressed = false;
114.     }
115. }
116.
117. // -----
118.
119. // Reset buttons:
120. // -----
121.
122. inline void ResetButton1(void)
123. {
124.     bt1_pressed = false;
125. }
126.
127. inline void ResetButton2(void)
128. {
129.     bt2_pressed = false;
130. }
131.
132. inline void ResetButton3(void)
133. {
134.     bt3_pressed = false;
135. }
136.

```

```

137.
138.     inline void ResetButton4(void)
139.     {
140.         bt4_pressed = false;
141.     }
142.
143.     // Check buttons:
144.     // -----
145.
146.     inline int IsPowerButtonPressed()
147.     {
148.         return bt1_pressed == true;
149.     }
150.
151.     inline int IsUpperButtonPressed(void)
152.     {
153.         return bt1_pressed == true;
154.     }
155.
156.     inline int IsLowerButtonPressed(void)
157.     {
158.         return bt2_pressed == true;
159.     }
160.
161.     inline int IsOkButtonPressed()
162.     {
163.         return bt3_pressed == true;
164.     }
165.
166.     inline int IsFaultSimulationButtonPressed()
167.     {
168.         return bt4_pressed == true;
169.     }
170.
171.     // OkButtonPressed:
172.     // -----
173.
174.     inline void ResetAllButtons(void)
175.     {
176.         ResetButton1();
177.         ResetButton2();
178.         ResetButton3();
179.         ResetButton4();
180.     }
181.
182. #endif // !BUTTONS_H

```

Szczegółowy opis kodu z pliku **Buttons.h**:

- Linie 9-10: Stworzenie makr reprezentujących wciśnięcie i zwolnienie przycisku.
- Linie 12-13: Stworzenie odpowiednich stałych czasu powiązanych z debouncingiem.
- Linie 18-21: Definicja czterech zmiennych logicznych reprezentujących wciśnięcie przycisków od RA1-RA4. Standardowo podczas odniesienia się pierwszy raz do tych zmiennych – mają przypisaną wartość false.
- Linie 28-31: Definicja funkcji sprawdzającej czy przycisk RA1 został wciśnięty ale bez debouncingu.
- Linie 33-36: Definicja funkcji sprawdzającej czy przycisk RA2 został wciśnięty ale bez debouncingu.
- Linie 38-41: Definicja funkcji sprawdzającej czy przycisk RA3 został wciśnięty ale bez debouncingu.
- Linie 43-46: Definicja funkcji sprawdzającej czy przycisk RA4 został wciśnięty ale bez debouncingu.

Linie 54-68: Definicja funkcji obsługującej debouncing dla przycisku RA1.
Linie 70-83: Definicja funkcji obsługującej debouncing dla przycisku RA2.
Linie 85-99: Definicja funkcji obsługującej debouncing dla przycisku RA3.
Linie 101-115: Definicja funkcji obsługującej debouncing dla przycisku RA4.

Linie 123-126: Definicja funkcji zmieniającej stan przycisku RA1 na zwolniony.
Linie 128-131: Definicja funkcji zmieniającej stan przycisku RA2 na zwolniony.
Linie 133-136: Definicja funkcji zmieniającej stan przycisku RA3 na zwolniony.
Linie 138-141: Definicja funkcji zmieniającej stan przycisku RA4 na zwolniony.

Linie 146-149: Definicja funkcji sprawdzającej wciśnięcie przycisku PowerButton.
Linie 151-154: Definicja funkcji sprawdzającej wciśnięcie przycisku UpperButton.
Linie 156-159: Definicja funkcji sprawdzającej wciśnięcie przycisku LowerButton.
Linie 161-164: Definicja funkcji sprawdzającej wciśnięcie przycisku OkButton.
Linie 166-169: Definicja funkcji sprawdzającej wciśnięcie przycisku FaultSimulationButton.

Linie 174-180: Definicja funkcji zmieniającej stan wszystkich przycisków od RA1 do RA4 na zwolniony.

c) Display.h

```
1. #ifndef DISPLAY_H
2. #define DISPLAY_H
3.
4. #include <xc.h>
5.
6. // Consts:
7. // -----
8.
9. enum ENDisplayControlBits
10. {
11.     RIGHT_DISPLAY_FLAG = 0x10
12. };
13.
14. // Variables:
15. // -----
16.
17. static volatile uint8_t left_digit      = 0;
18. static volatile uint8_t right_digit     = 0;
19. static volatile _Bool is_display_enabled = false;
20.
21.
22. // Utils:
23. // -----
24.
25. inline void SetValueOnDisplay(uint8_t arg_left_dig, uint8_t arg_right_dig)
26. {
27.     left_digit  = arg_left_dig;
28.     right_digit = arg_right_dig;
29. }
30.
31. static uint8_t Display7s(uint8_t v)
32. {
33.     switch(v)
34.     {
35.         case 0: return 0xEE;
36.         case 1: return 0x28;
37.         case 2: return 0xCD;
38.         case 3: return 0x6D;
39.         case 4: return 0x2B;
40.         case 5: return 0x67;
41.         case 6: return 0xE7;
42.         case 7: return 0x2C;
43.         case 8: return 0xEF;
44.         case 9: return 0x6F;
45.         case 10: return 0xAF;
46.         case 11: return 0xE3;
47.         case 12: return 0xC6;
48.         case 13: return 0xE9;
49.         case 14: return 0xC7;
50.         case 15: return 0x87;
51.         default: return 0;
52.     }
53. }
54.
55. inline void DisableDisplay(void)
56. {
57.     is_display_enabled = false;
58. }
59.
60. inline void EnableDisplay(void)
61. {
62.     is_display_enabled = true;
63. }
64.
65. // -----
66.
67. // private:
```

```

68. inline void Private_ShowDisplay(void)
69. {
70.     volatile static _Bool refresh_flag = true;
71.
72.     if(refresh_flag)
73.         PORTB = Display7s(left_digit);
74.     else
75.         PORTB = (uint8_t)(Display7s(right_digit) | RIGHT_DISPLAY_FLAG);
76.
77.     refresh_flag = ~refresh_flag;
78. }
79.
80. #endif // !DISPLAY_H
81.

```

Szczegółowy opis kodu z pliku **Display.h**:

- Linie 9-12: Definicja stałych w enum - służących do kontroli sterowania wyświetlacza siedmiosegmentowego.
- Linie 17-18: Definicja zmiennych reprezentujących aktualnie wyświetlane cyfry na wyświetlaczu siedmiosegmentowym.
- Linia 19: Definicja zmiennej logicznej włączającej/wyłączającej wyświetlacz siedmiosegmentowy.
- Linie 25-29: Definicja funkcji zmieniającej aktualną wartość na wyświetlaczu siedmiosegmentowym.
- Linie 31-53: Definicja funkcji zwracającej odpowiedni sygnał dla mikrokontrolera na podstawie argumentu reprezentującego cyfrę na wyświetlaczu siedmiosegmentowym. Funkcja przyjmuje za argument cyfrę na wyświetlaczu siedmiosegmentowym i zwraca wartość, która po wprowadzeniu do portu B mikrokontrolera – wyświetli cyfrę wprowadzoną w argumencie funkcji na wyświetlaczu siedmiosegmentowym.
- Linie 55-58: Definicja funkcji wyłączającej wyświetlacz siedmiosegmentowy.
- Linie 60-63: Definicja funkcji włączającej wyświetlacz siedmiosegmentowy.
- Linie 68-78: Definicja funkcji służącej do obsługi wyświetlania cyfr na wyświetlaczu siedmiosegmentowym.

d) DisplayTimer.h

```
1.
2. #ifndef DISPLAY_TIMER_H
3. #define DISPLAY_TIMER_H
4.
5. #include <xc.h>
6.
7. static volatile _Bool is_display_timer_enabled = false;
8.
9. inline void DisableDisplayTimer(void)
10. {
11.     is_display_timer_enabled = false;
12. }
13.
14. inline void EnableDisplayTimer(void)
15. {
16.     is_display_timer_enabled = true;
17. }
18.
19. inline int IsDisplayTimerFinished(void)
20. {
21.     return (left_digit == 0) && (right_digit == 0);
22. }
23.
24. // -----
25.
26. // private:
27. inline void Private_DecrementDisplayTimer(void)
28. {
29.     if(right_digit != 0)
30.     {
31.         --right_digit;
32.     }
33.     else
34.     {
35.         if(left_digit != 0)
36.         {
37.             --left_digit;
38.             right_digit = 9;
39.         }
40.     }
41. }
42.
43. #endif // !DISPLAY_TIMER_H
44.
45.
```

Szczegółowy opis kodu z pliku **DisplayTimer.h**:

- Linia 5: Załączenie niezbędnych plików nagłówkowych.
- Linia 7: Definicja zmiennej sterującej włączeniem/wyłączeniem stopera korzystającego z wyświetlacza siedmiosegmentowego.
- Linie 9-12: Definicja funkcji wyłączającej stoper korzystający z wyświetlacza siedmiosegmentowego.
- Linie 14-17: Definicja funkcji włączającej stoper korzystający z wyświetlacza siedmiosegmentowego.
- Linie 19-22: Definicja funkcji sprawdzającej czy stoper korzystający z wyświetlacza siedmiosegmentowego - zakończył odliczanie.
- Linie 27-41: Definicja funkcji zmniejszającej wartość na stoperze korzystającym z wyświetlacza siedmiosegmentowego.

e) Error.h

```
1.
2. #ifndef ERRORS_H
3. #define ERRORS_H
4.
5. #include <xc.h>
6.
7. // Enums:
8. // -----
9.
10. enum ENErrorCodes
11. {
12.     NO_ERROR = 0,
13.     ERROR_FOR_STATE_DISABLED = 1,
14.     ERROR_FOR_STATE_READY = 2,
15.     ERROR_FOR_STATE_HEATING = 3,
16.     ERROR_FOR_STATE_INSERT_CARTRIDGE = 4,
17.     ERROR_FOR_STATE_INSERT_PRODUCTS = 5,
18.     ERROR_FOR_STATE_BAKING = 6,
19.     ERROR_FOR_STATE_GET_PRODUCTS = 7,
20.     ERROR_FOR_STATE_PREPARE_TO_CLEANING = 8,
21.     ERROR_FOR_STATE_CLEANING = 9
22. };
23.
24. static uint8_t error_code = NO_ERROR;
25.
26. // Error utils:
27. // -----
28.
29. static uint8_t GetErrorFromState(STATE_t state)
30. {
31.     switch(state)
32.     {
33.         case STATE_DISABLED:
34.             return ERROR_FOR_STATE_DISABLED;
35.         break;
36.
37.         case STATE_READY:
38.             return ERROR_FOR_STATE_READY;
39.         break;
40.
41.         case STATE_HEATING:
42.             return ERROR_FOR_STATE_HEATING;
43.         break;
44.
45.         case STATE_INSERT_CARTRIDGE:
46.             return ERROR_FOR_STATE_INSERT_CARTRIDGE;
47.         break;
48.
49.         case STATE_INSERT_PRODUCTS:
50.             return ERROR_FOR_STATE_INSERT_PRODUCTS;
51.         break;
52.
53.         case STATE_BAKING:
54.             return ERROR_FOR_STATE_BAKING;
55.         break;
56.
57.         case STATE_GET_PRODUCTS:
58.             return ERROR_FOR_STATE_GET_PRODUCTS;
59.         break;
60.
61.         case STATE_PREPARE_TO_CLEANING:
62.             return ERROR_FOR_STATE_PREPARE_TO_CLEANING;
63.         break;
64.
65.         case STATE_CLEANING:
66.             return ERROR_FOR_STATE_CLEANING;
67.         break;
68.
69.         default:
```



```

70.         return NO_ERROR;
71.     break;
72. }
73. }
74.
75.
76. inline void CheckThatFaultAppeared(void)
77. {
78.     if(IsFaultSimulationButtonPressed())
79.     {
80.         error_code = GetErrorFromState(current_state_name);
81.
82.         ChangeMachineState(STATE_ERROR, StateErrorFn);
83.
84.         action = ACTION_ENTRY;
85.     }
86. }
87.
88. inline void ResetError(void)
89. {
90.     error_code = NO_ERROR;
91. }
92.
93. #endif // !ERRORS_H
94.
95.

```

Szczegółowy opis kodu z pliku **Error.h**:

- Linie 10-22: Definicja stałych w enum – służących do reprezentacji kodów błędów dla odpowiednich stanów.
- Linia 24: Definicja zmiennej reprezentującej aktualny kod błędu.
- Linie 29-73: Definicja funkcji przyjmującej za argument stan i zwracającej kod błędu dla tego stanu.
- Linie 76-86: Definicja funkcji sprawdzającej czy wystąpiła awaria sprzętowa w jednym ze stanów. Jeśli błąd wystąpił to dochodzi do zmienienia stanu na STATE_ERROR i wyświetlenie kodu błędu na wyświetlaczu siedmiosegmentowym.
- Linie 88-91: Definicja funkcji zmieniająca kod błędu na brak błędu.

f) Leds.h

```
1. #ifndef LEDS_H
2. #define LEDS_H
3.
4. #include <xc.h>
5.
6. // Macros for leds:
7. // -----
8.
9. #define ROT_R1(x)    (((x) >> 1) | ((x) << 7))
10. #define ROT_L1(x)    (((x) << 1) | ((x) >> 7))
11.
12. #define NUMBER_OF_LEDS 8
13.
14. // Enums for leds:
15. // -----
16.
17. enum ENLedFlags
18. {
19.     FIRST_LED_ON = 0x01,
20.     ALL_LEDS_OFF = 0x00,
21.     ALL_LEDS_ON  = 0xFF
22. };
23.
24. // Functions:
25. // -----
26.
27. inline void AllLedsOn(void)
28. {
29.     PORTB = ALL_LEDS_ON;
30. }
31.
32. inline void AllLedsOff(void)
33. {
34.     PORTB = ALL_LEDS_OFF;
35. }
36.
37. inline void LampOn(void)
38. {
39.     PORTAbits.RA0 = 1;
40. }
41.
42. inline void LampOff(void)
43. {
44.     PORTAbits.RA0 = 0;
45. }
46.
47.
48. #endif // !LEDS_H
49.
```

Szczegółowy opis kodu z pliku **Leds.h**:

- Linie 9-10: Definicja makr ROT_R1 i ROT_L1 wykonujących obrót bitowy.
- Linia 12: Stworzenie stałej odnoszącej się do ilości diod LED na płytce.
- Linie 17-22: Definicja stałych dla portu B reprezentujących: pierwszy włączony LED (RB0), wszystkie LEDy wyłączone (RB0-RB7), wszystkie LEDy włączone (RB0 – RB7).
- Linie 27-30: Definicja funkcji włączającej wszystkie LEDy (RB0 – RB7).
- Linie 32-35: Definicja funkcji wyłączającej wszystkie LEDy (RB0 – RB7).
- Linie 37-40: Definicja funkcji włączająca lampę RA0.
- Linie 42-45: Definicja funkcji wyłączającej lampę RA0.

g) Microcontroller.h

```
1.
2. #ifndef MICROCONTROLLER_H
3. #define MICROCONTROLLER_H
4.
5. #include <xc.h>
6. #include "Timers.h"
7. #include "Leds.h"
8. #include "Display.h"
9. #include "DisplayTimer.h"
10. #include "Other.h"
11.
12. // private:
13. // -----
14.
15. inline void Private_InitMicrocontrollerTimer0(void)
16. {
17.     OPTION_REG = 0b11010111;
18.     TMR0 = 13;
19. }
20.
21. inline void Private_InitMicrocontrollerTimer1(void)
22. {
23.     T1CON = 0b00100101;
24.     TMR1H = TMR1L = 0;
25. }
26.
27.
28. // Microcontroller config:
29. // -----
30.
31. static void InitPeripherals() // Konfiguracja mikrokontrolera
32. {
33.     TRISA=0xFE; // 0xFE = 11111110 - jedyne 0 odnosi sie do lampki
34.     TRISB=0x00;
35.
36.     Private_InitMicrocontrollerTimer0();
37.     Private_InitMicrocontrollerTimer1();
38.
39.     // Interrupt setup
40.     INTCONbits.TMR0IE = 1; // enable interrupts for timer 0
41.     PIE1bits.TMR1IE = 1; // enable interrupts for timer 1
42.     INTCONbits.PEIE = 1; // enable peripheral interrupts
43.
44.     ei();
45. }
46.
47. void __interrupt() isr(void)
48. {
49.     // only process timer0-triggered interrupts
50.     if(INTCONbits.TMR0IE && INTCONbits.TMR0IF)
51.     {
52.         Private_DebounceButton1();
53.         Private_DebounceButton2();
54.         Private_DebounceButton3();
55.         Private_DebounceButton4();
56.
57.         DECREMENT_TIMER(soft_timer_1);
58.
59.         DECREMENT_TIMER(soft_timer_2);
60.
61.         DECREMENT_TIMER(soft_timer_3);
62.
63.
64.         // -----
65.         // For loading:
66.         if(is_loading_anim_enabled)
67.         {
68.             Private_UpdateLoadingAnim();
69.         }
```

```

70.      // -----
71.
72.
73.      // -----
74.      // For Switching/showing displays:
75.      if(is_display_enabled)
76.      {
77.          Private_ShowDisplay();
78.      }
79.      // -----
80.
81.
82.      INTCONbits.TMR0IF = 0; // clear this interrupt condition
83.      TMR0 = 13;
84.  }
85.
86.  // only process timer1-triggered interrupts
87.  if(PIE1bits.TMR1IE && PIR1bits.TMR1IF)
88.  {
89.      // -----
90.      // For display timer:
91.      if(is_display_enabled && is_display_timer_enabled)
92.      {
93.          Private_DecrementDisplayTimer();
94.      }
95.      // -----
96.
97.      PIR1bits.TMR1IF = 0; // clear this interrupt condition
98.  }
99. }
100.
101. #endif // !MICROCONTROLLER_H
102.

```

Szczegółowy opis kodu z pliku **Microcontroller.h**:

- Linie 5-10: Załączenie niezbędnych plików nagłówkowych.
- Linia 15-19: Inicjalizacja Timera 0 w mikrokontrolerze.
- Linia 21-25: Inicjalizacja Timera 1 w mikrokontrolerze.

- Linia 31-45: Definicja funkcji konfigurującej mikrokontroler.
- Linia 47-99: Definicja funkcji obsługującej przerwania.

- Linia 49-89: Obsługa przerwania wyzwalanego przez Timer 0.
- Linie 52-55: Zastosowanie debouncingu dla przycisków RA1 – RA4.
- Linie 57-61: Dekrementacja soft timerów.
- Linie 66-69: Odświeżanie animacji ładowania jeśli jest włączona.
- Linie 75-78: Wyświetlanie cyfr na wyświetlaczu siedmiosegmentowym jeśli wyświetlacz siedmiosegmentowy jest włączony.
- Linia 82: Wyczyszczenie warunku przerwania dla Timera 0.

- Linia 86-98: Obsługa przerwania wyzwalanego przez Timer 1.
- Linia 91-94: Dekrementacja stopera korzystającego z wyświetlacza siedmiosegmentowego w przypadku gdy stoper dla wyświetlacza siedmiosegmentowego jest włączony.
- Linia 97: Wyczyszczenie warunku przerwania dla Timera 1.

h) Other.h

```
1.
2. #ifndef OTHER_H
3. #define OTHER_H
4.
5. #include <xc.h>
6. #include "Leds.h"
7.
8. #define ANIM_TICK_PERIOD_FOR_LOADING      5    // co ile tickow przesuwa sie
   lampka
9.
10. volatile static _Bool    is_loading_anim_enabled = false;
11. volatile static uint8_t  anim_tick_period = ANIM_TICK_PERIOD_FOR_LOADING;
12. volatile static uint8_t  anim_step = 0;
13. volatile static uint8_t  anim_tick = 0;
14.
15. // Bell abstraction:
16. // -----
17.
18. // Czyli wszystkie ledy migaja
19. inline void RingTheBell(void)
20. {
21.     PORTB = (uint8_t)~PORTB;
22. }
23.
24. // Loading abstraction:
25. // -----
26.
27. // Czyli ledy zapalaja sie po kolei
28. inline void Loading(void)
29. {
30.     static uint8_t last_step = ALL_LEDS_ON; // wymusza rysowanie przy starcie
31.
32.     if(anim_step != last_step)
33.     {
34.         last_step = anim_step;
35.
36.         if(anim_step >= NUMBER_OF_LEDS)
37.         {
38.             // Caly pasek zapalony = gotowe
39.             AllLedsOn();
40.         }
41.         else
42.         {
43.             // Zapalanie od RB7 do RB0:
44.             // Dla anim_step = 0: 0xFF << (8 - 1 - 0) -> 0xFF << 7 -> 0b10000000
45.             // Dla anim_step = 1: 0xFF << (8 - 1 - 1) -> 0xFF << 6 -> 0b11000000
46.             PORTB = (uint8_t)(ALL_LEDS_ON << (NUMBER_OF_LEDS - 1 - anim_step));
47.
48.
49.             // Ta wersja przesuwała w lewo od RB0 do RB7:
50.             // PORTB = (uint8_t)((1 << (anim_step + 1)) - 1);
51.         }
52.     }
53.
54. }
55.
56. // Loading anim functions:
57. // -----
58.
59. inline void EnableLoadingAnim(void)
60. {
61.     is_loading_anim_enabled = true;
62. }
63.
64. inline void DisableLoadingAnim(void)
65. {
66.     is_loading_anim_enabled = false;
67. }
68.
```

```

69.
70. // Cleaning abstractions:
71. // -----
72.
73. inline void MotorOn(void)
74. {
75.     PORTBbits.RB1 = 1;
76. }
77.
78. inline void MotorOff(void)
79. {
80.     PORTBbits.RB1 = 0;
81. }
82.
83. inline void PrepareToCleaning(void)
84. {
85.     PORTB = (uint8_t) (ROT_L1(PORTB));
86. }
87.
88. // Heating abstractions:
89. // -----
90.
91. inline void HeaterOn(void)
92. {
93.     PORTBbits.RB7 = 1;
94. }
95.
96. inline void HeaterOff(void)
97. {
98.     PORTBbits.RB7 = 0;
99. }
100.
101. // Fan abstractions:
102. // -----
103.
104. inline void FanOn(void)
105. {
106.     PORTBbits.RB6 = 1;
107. }
108.
109. inline void FanOff(void)
110. {
111.     PORTBbits.RB6 = 0;
112. }
113.
114. // Water valve abstraction:
115. // -----
116.
117. inline void OpenWaterValve(void)
118. {
119.     PORTBbits.RB4 = 1;
120. }
121.
122. inline void CloseWaterValve(void)
123. {
124.     PORTBbits.RB4 = 0;
125. }
126.
127. // Drain valve abstraction:
128. // -----
129.
130. inline void OpenDrainValve(void)
131. {
132.     PORTBbits.RB5 = 1;
133. }
134.
135. inline void CloseDrainValve(void)
136. {
137.     PORTBbits.RB5 = 0;
138. }
139.

```

```

140.      // -----
141.
142.      // private:
143.
144.      inline void Private_UpdateLoadingAnim(void)
145.      {
146.          if(anim_tick < anim_tick_period)
147.          {
148.              ++anim_tick;
149.          }
150.          else
151.          {
152.              anim_tick = 0;
153.
154.              if(anim_step < NUMBER_OF_LEDS)
155.                  ++anim_step;    // flaga dla main()
156.          }
157.      }
158.
159.      #endif // !OTHER_H
160.

```

Szczegółowy opis kodu z pliku **Other.h**:

- Linie 5-6: Załączenie niezbędnych plików nagłówkowych.
- Linia 8: Stworzenie stałej reprezentującej co ile ticków przesuwa się LED w animacji ładowania.
- Linia 10: Definicja zmiennej sterującej włączeniem/wyłączeniem animacji ładowania.
- Linie 12-13: Definicja zmiennych związanych z animacją ładowania.
- Linia 19-22: Definicja funkcji sterującej dzwonkiem.
- Linie 28-54: Definicja funkcji sterującej animacją ładowania. Funkcja zapala po kolei LEDy od RB0 do RB7.
- Linie 59-62: Definicja funkcji włączającej animację ładowania.
- Linie 64-67: Definicja funkcji wyłączającej animację ładowania.

- Linie 73-76: Definicja funkcji włączającej silnik.
- Linie 78-81: Definicja funkcji wyłączającej silnik.

- Linie 83-86: Definicja funkcji przygotowującej do czyszczenia pieca.

- Linie 91-94: Definicja funkcji włączającej grzałki.
- Linie 96-99: Definicja funkcji wyłączającej grzałki.

- Linie 104-107: Definicja funkcji włączającej wentylator.
- Linie 109-112: Definicja funkcji wyłączającej wentylator.

- Linie 117-120: Definicja funkcji otwierającej zawór wody.
- Linie 122-125: Definicja funkcji zamykającej zawór wody.

- Linie 130-133: Definicja funkcji otwierającej zawór odpływowy.
- Linie 122-125: Definicja funkcji zamykającej zawór odpływowy.
- Linie 144-157: Definicja funkcji odświeżającej animację ładowania.

i) Programs.h

```
1. #ifndef PROGRAMS_H
2. #define PROGRAMS_H
3.
4. #include <xc.h>
5. #include "Buttons.h"
6.
7. // Consts:
8.
9. #define PROGRAM_SETTED      true
10. #define PROGRAM_NOT_SETTED  false
11.
12. // Programs:
13. // -----
14.
15. typedef enum
16. {
17.     PROGRAM_0_BUNS_FULL          = 0x01, // Bulki - caly piec
18.     PROGRAM_1_BUNS_HALF          = 0x02, // Bulki - pol pieca
19.     PROGRAM_2_COOKIES_FULL       = 0x04, // Ciastka - caly piec
20.     PROGRAM_3_COOKIES_HALF       = 0x08, // Ciastka - pol pieca
21.     PROGRAM_4_BREAD_2_TO_20      = 0x10, // Chleb od 2 do 20
22.     PROGRAM_5_CASSEROLE_OR_SWEET_BUN = 0x20, // Zapiekanka / drozdzowka
23.     PROGRAM_6_CLEANING_THE_MACHINE = 0x40, // Mycie pieca
24.     PROGRAM_7_QUIT                = 0x80  // Wyjscie
25. } PROGRAM_t;
26.
27.
28. // Initial program:
29. // -----
30.
31. static PROGRAM_t program = PROGRAM_0_BUNS_FULL;    // okreslamy poczatkowy
    program
32.
33.
34. // Program times:
35. // -----
36.
37. #define PROGRAM_0_HEATING_TIME    NUMBER_OF_LEDS * 8
38. #define ANIM_TICK_PERIOD_FOR_PROGRAM_0    PROGRAM_0_HEATING_TIME/NUMBER_OF_LEDS
39.
40. #define PROGRAM_1_HEATING_TIME    NUMBER_OF_LEDS * 7
41. #define ANIM_TICK_PERIOD_FOR_PROGRAM_1    PROGRAM_1_HEATING_TIME/NUMBER_OF_LEDS
42.
43. #define PROGRAM_2_HEATING_TIME    NUMBER_OF_LEDS * 6
44. #define ANIM_TICK_PERIOD_FOR_PROGRAM_2    PROGRAM_2_HEATING_TIME/NUMBER_OF_LEDS
45.
46. #define PROGRAM_3_HEATING_TIME    NUMBER_OF_LEDS * 9
47. #define ANIM_TICK_PERIOD_FOR_PROGRAM_3    PROGRAM_3_HEATING_TIME/NUMBER_OF_LEDS
48.
49. #define PROGRAM_4_HEATING_TIME    NUMBER_OF_LEDS * 10
50. #define ANIM_TICK_PERIOD_FOR_PROGRAM_4    PROGRAM_4_HEATING_TIME/NUMBER_OF_LEDS
51.
52. #define PROGRAM_5_HEATING_TIME    NUMBER_OF_LEDS * 8
53. #define ANIM_TICK_PERIOD_FOR_PROGRAM_5    PROGRAM_5_HEATING_TIME/NUMBER_OF_LEDS
54.
55. // Programs utils:
56. // -----
57.
58. static uint8_t GetHeatingTime(PROGRAM_t program)
59. {
60.     switch(program)
61.     {
62.         case PROGRAM_0_BUNS_FULL:
63.             return PROGRAM_0_HEATING_TIME;
64.
65.         case PROGRAM_1_BUNS_HALF:
66.             return PROGRAM_1_HEATING_TIME;
67.
68.         case PROGRAM_2_COOKIES_FULL:
```



```

69.         return PROGRAM_2_HEATING_TIME;
70.
71.     case PROGRAM_3_COOKIES_HALF:
72.         return PROGRAM_3_HEATING_TIME;
73.
74.     case PROGRAM_4_BREAD_2_TO_20:
75.         return PROGRAM_4_HEATING_TIME;
76.
77.     case PROGRAM_5_CASSEROLE_OR_SWEET_BUN:
78.         return PROGRAM_5_HEATING_TIME;
79.
80.     default:
81.         return 0;
82. }
83. }
84.
85.
86.
87. static uint8_t GetAnimTickPeriodForHeating(PROGRAM_t program)
88. {
89.     switch(program)
90.     {
91.         case PROGRAM_0_BUNS_FULL:
92.             return ANIM_TICK_PERIOD_FOR_PROGRAM_0;
93.
94.         case PROGRAM_1_BUNS_HALF:
95.             return ANIM_TICK_PERIOD_FOR_PROGRAM_1;
96.
97.         case PROGRAM_2_COOKIES_FULL:
98.             return ANIM_TICK_PERIOD_FOR_PROGRAM_2;
99.
100.            case PROGRAM_3_COOKIES_HALF:
101.                return ANIM_TICK_PERIOD_FOR_PROGRAM_3;
102.
103.            case PROGRAM_4_BREAD_2_TO_20:
104.                return ANIM_TICK_PERIOD_FOR_PROGRAM_4;
105.
106.            case PROGRAM_5_CASSEROLE_OR_SWEET_BUN:
107.                return ANIM_TICK_PERIOD_FOR_PROGRAM_5;
108.
109.            default:
110.                return 0;
111.        }
112.    }
113.
114.    inline void SelectProgram0(void)
115.    {
116.        PORTBbits.RB0 = 1;
117.        program = PROGRAM_0_BUNS_FULL;
118.    }
119.
120.    //
121.    // Gdy ktorys z przyciskow zostal wcisniety podczas ustawiania programu
122.    // czyli RA1 lub RA2 lub RA3 to zwracana jest wartosc true (PROGRAM_SETTED)
123.    // oznaczajacy ze ustawiono jakis program przez co mozna potem go odebrac
124.    // i np zresetowac timer lub zresetowac przyciski z innego miejsca w
    programie
125.    // (mozna to zauwazyc w main())
126.    //
127.    // Jezeli nie wcisnieto przycisku to zwraca false (PROGRAM_NOT_SETTED)
128.    //
129.    inline _Bool SetProgram(void)
130.    {
131.        if(IsUpperButtonPressed())
132.        {
133.            PORTB = (uint8_t)(ROT_R1(PORTB));
134.            program = (uint8_t)(ROT_R1(program));
135.            return PROGRAM_SETTED;
136.        }
137.        else if(IsLowerButtonPressed())
138.        {

```

```

139.         PORTB = (uint8_t) (ROT_L1(PORTB));
140.         program = (uint8_t) (ROT_L1(program));
141.         return PROGRAM_SETTED;
142.     }
143.     //
144.     // Dopiero jak user wcisnie przycisk OK czyli RA3
145.     // to wtedy program jest zatwierdzany i przechodzimy
146.     // do ACTION_EXIT
147.     //
148.     else if(IsOkButtonPressed())
149.     {
150.         action = ACTION_EXIT;
151.         return PROGRAM_SETTED;
152.     }
153.
154.     return PROGRAM_NOT_SETTED;
155. }
156.
157.
158. #endif // !PROGRAMS_H
159.

```

Szczegółowy opis kodu z pliku **Programs.h**:

- Linie 4-5: Załączenie niezbędnych plików nagłówkowych.
- Linia 9-10: Stworzenie stałych reprezentujących komunikat o ustawieniu programu lub o jego nieustawieniu.
- Linie 15-25: Definicja stałych reprezentujących poszczególne programy pracy (zastosowano metodę 1 z n).
- Linia 31: Definicja zmiennej reprezentującej aktualny program. Podczas pierwszego odwołania do zmiennej – do zmiennej zostanie przypisana wartość PROGRAM_0_BUNS_FULL.
- Linie 37-53: Stworzenie makr reprezentujących poszczególne czasy nagrzewania dla programów od 0 do 5.
- Linie 58-83: Definicja funkcji przyjmującej za argument program, która zwraca odpowiednio dobrany czas nagrzewania.
- Linie 87-112: Definicja funkcji przyjmującej za argument program, która zwraca odpowiednio dobrany tick dla animacji ładowania.
- Linie 114-118: Definicja funkcji ustawiającej program na PROGRAM_0_BUNS_FULL czyli cały piec bułek.
- Linie 129-155: Definicja funkcji służącej do sterowania wyborem programu.

j) StatesAndActions.h

```
1. #ifndef STATES_AND_ACTIONS_H
2. #define STATES_AND_ACTIONS_H
3.
4. #include <xc.h>
5.
6. // State machine - state names:
7. // -----
8.
9. typedef enum
10. {
11.     STATE_POWERUP,
12.     STATE_DISABLED,
13.     STATE_READY,
14.     STATE_HEATING,
15.     STATE_INSERT_CARTRIDGE,
16.     STATE_INSERT_PRODUCTS,
17.     STATE_BAKING,
18.     STATE_GET_PRODUCTS,
19.     STATE_PREPARE_TO_CLEANING,
20.     STATE_CLEANING,
21.     STATE_ERROR
22. } STATE_t;
23.
24. // State machine - state function type:
25. // -----
26.
27. typedef void (*pStateFn)(void);
28.
29. // State machine - state functions declarations:
30. // -----
31.
32. void StatePowerUpFn(void);
33. void StateDisabledFn(void);
34. void StateReadyFn(void);
35. void StateHeatingFn(void);
36. void StateInsertCartridgeFn(void);
37. void StateInsertProductsFn(void);
38. void StateBakingFn(void);
39. void StateGetProductsFn(void);
40. void StatePrepareToCleaningFn(void);
41. void StateCleaningFn(void);
42. void StateErrorFn(void);
43.
44.
45. // State machine - state actions:
46. // -----
47.
48. typedef enum
49. {
50.     ACTION_ENTRY,           // akcja wykonywana 1x podczas wejścia do stanu
51.     ACTION_DO,              //
52.     ACTION_EXIT,           // akcja wykonywana 1x podczas wyjścia ze stanu
53. } ACTION_t;
54.
55.
56. // Initial state:
57. // -----
58.
59. static STATE_t current_state_name = STATE_POWERUP; // określamy stan
    początkowy
60. static pStateFn CurrentStateFunction = StatePowerUpFn; //
61.
62. // Initial action:
63. // -----
64.
65. static ACTION_t action = ACTION_ENTRY; // określamy akcję początkową
66.
67. // Change state:
68. // -----
```

```

69.
70. inline void ChangeMachineState(STATE_t state_name, pStateFn state_function)
71. {
72.     current_state_name      = state_name;
73.     CurrentStateFunction = state_function;
74. }
75.
76. #endif // !STATES_AND_ACTIONS_H
77.

```

Szczegółowy opis kodu z pliku **StatesAndActions.h**:

- Linia 4: Załączenie niezbędnych plików nagłówkowych.
- Linie 9-22: Definicja poszczególnych stanów w maszynie stanowej.
- Linia 27: Definicja nowego typu danych o nazwie pStateFn, który jest wskaźnikiem na funkcję.
- Linie 32-42: Deklaracja funkcji, które reprezentują każdy stan maszyny stanowej.
- Linie 48-53: Definicja możliwych akcji (entry, do, exit).
- Linie 59-60: Definicja zmiennych reprezentujących aktualny stan maszyny stanowej.
- Linia 65: Definicja zmiennej określającej akcję początkową.
- Linie 70-74: Funkcja służąca do zmiany aktualnego stanu maszyny stanowej.

k) Timers.h

```
1. #ifndef TIMERS_H
2. #define TIMERS_H
3.
4. #include <xc.h>
5. #include "Programs.h"
6. #include "Other.h"
7.
8. // Macros:
9. // -----
10.
11. #define DECREMENT_TIMER(some_timer) \
12.     if(some_timer) --some_timer;
13.
14.
15. // Consts:
16. // -----
17.
18. #define PREPARE_TO_CLEANING_TIME          40
19. #define PERIOD_FOR_ROTATIONS              3
20.
21. #define LOADING_TIME                      40
22.
23. #define TIME_OUT                          60
24.
25. #define PERIOD_FOR_BELL                   5
26.
27.
28. // Soft timers:
29. // -----
30.
31. volatile static uint8_t soft_timer_1;
32. volatile static uint8_t soft_timer_2;
33. volatile static uint8_t soft_timer_3;
34.
35. // Timers setters:
36. // -----
37.
38. inline void SetSoftTimer1ForPrepareToCleaning()
39. {
40.     soft_timer_1 = PREPARE_TO_CLEANING_TIME;
41. }
42.
43. inline void SetSoftTimer2ForRotations()
44. {
45.     soft_timer_2 = PERIOD_FOR_ROTATIONS;
46. }
47.
48. inline void SetSoftTimer2ForBell()
49. {
50.     soft_timer_2 = PERIOD_FOR_BELL;
51. }
52.
53. inline void SetSoftTimer3ForLoading()
54. {
55.     anim_tick_period = ANIM_TICK_PERIOD_FOR_LOADING;
56.     soft_timer_3      = LOADING_TIME;
57.     anim_step         = 0;
58. }
59.
60. inline void SetSoftTimer3ForHeating(PROGRAM_t program)
61. {
62.     anim_tick_period = GetAnimTickPeriodForHeating(program);
63.     soft_timer_3      = GetHeatingTime(program);
64.     anim_step         = 0;
65. }
66.
67. inline void SetSoftTimer3ForTimeOut(void)
68. {
69.     soft_timer_3      = TIME_OUT;
```

```

70. }
71.
72.
73. #endif // !TIMERS_H
74.

```

Szczegółowy opis kodu z pliku **Timers.h**:

- Linie 4-6: Załączenie niezbędnych plików nagłówkowych.
- Linie 11-12: Definicja makra służącego do dekrementacji stoperów.
- Linie 18-25: Stworzenie stałych czasowych dla odpowiednich wydarzeń.
- Linie 31-33: Definicja zmiennych reprezentujących stopery: `soft_timer_1`, `soft_timer_2`, `soft_timer_3`.
- Linie 38-41: Definicja funkcji, która ustawia stoper `soft_timer_1` na czas przygotowania do czyszczenia.
- Linie 43-46: Definicja funkcji, która ustawia stoper `soft_timer_2` na czas obrotu ramienia natryskowego.
- Linie 48-51: Definicja funkcji, która ustawia stoper `soft_timer_2` na czas dla sygnału dźwiękowego (dzwonek).
- Linie 53-58: Definicja funkcji, która ustawia stoper `soft_timer_3` na czas do animacji ładowania.
- Linie 60-65: Definicja funkcji, która przyjmuje za argument program i na jego podstawie ustawia stoper `soft_timer_3` na odpowiedni czas nagrzewania.
- Linie 67-70: Definicja funkcji, która ustawia limit czasu bezczynnego oczekiwania na stoperze `soft_timer_3`.

System jest obsługiwany za pomocą czterech przycisków od **RA1** do **RA4**, gdzie:

- RA1 jest przyciskiem PowerButton oraz przyciskiem zmieniającym program na wyższy (UpperButton),
- RA2 jest przyciskiem zmieniającym program na niższy program (LowerButton),
- RA3 jest przyciskiem zatwierdzającym (OkButton),
- RA4 jest przyciskiem symulującym awarię (FaultSimulationButton).

Wyróżniamy również następujące programy pracy pieca:

- PROGRAM_0_BUNS_FULL – Bułki – cały piec (0x01),
- PROGRAM_1_BUNS_HALF – Bułki – pół pieca (0x02),
- PROGRAM_2_COOKIES_FULL – Ciastka – cały piec (0x04),
- PROGRAM_3_COOKIES_HALF – Ciastka – pół pieca (0x08),
- PROGRAM_4_BREAD_2_TO_20 – Chleb – od dwóch do dwudziestu sztuk (0x10),
- PROGRAM_5_CASSEROLE_OR_SWEET_BUN – Zapiekanka lub drożdżówka (0x20),
- PROGRAM_6_CLEANING_THE_MACHINE – Mycie pieca (0x40),
- PROGRAM_7_QUIT – Wyjście (0x80).

Każdemu programowi przypisana jest unikalna wartość zakodowana metodą „1 z n” (**one-hot**), co umożliwia proste przełączanie między programami przy użyciu operacji **obrotu bitowego ROT_R1(X)** lub **ROT_L1(X)**.

W czasie pracy systemu może dojść do pewnych awarii, które sygnalizowane są odpowiednim kodem na wyświetlaczu siedmiosegmentowym. Kody awarii prezentują się następująco:

- NO_ERROR – Brak usterki (0),
- ERROR_FOR_STATE_DISABLED – Kod błędu dla stanu zablokowanego (1),
- ERROR_FOR_STATE_READY - Kod błędu dla stanu gotowości (2),
- ERROR_FOR_STATE_HEATING – Kod błędu dla stanu nagrzewania (3),
- ERROR_FOR_STATE_INSERT_CARTRIDGE – Kod błędu dla stanu umieszczania wkładu czyszczącego (4),
- ERROR_FOR_STATE_INSERT_PRODUCTS – Kod błędu dla stanu wkładania produktów do odpieku (5),
- ERROR_FOR_STATE_BAKING – Kod błędu dla stanu pieczenia (6),
- ERROR_FOR_STATE_GET_PRODUCTS – Kod błędu dla stanu odbioru produktów (7),
- ERROR_FOR_STATE_PREPARE_TO_CLEANING – Kod błędu dla stanu przygotowania do czyszczenia pieca (8),
- ERROR_FOR_STATE_CLEANING – Kod błędu dla stanu czyszczenia pieca (9).